

Reporte de Avance Proyecto Final – Sistemas Operativos

Integrantes del equipo: Fidel Ríos Salazar y
Oswaldo Munoz

Materia: Sistemas Operativos

Nombre Docente: Roberto Carlos Vaquera

Universidad Tecmilenio

Segundo Semestre - Bloque 1

Enlace a repo GitHub: [elfidelito27-
rgb/Proyecto SistemasOperativos](https://github.com/elfidelito27-rgb/Proyecto_SistemasOperativos)

Índice

Introducción	#
Diseño del Sistema	#
• 2.1 Arquitectura de Tres Capas	#
• 2.2 Decisiones de Diseño (TCP/IP vs UDP)	#
Implementación de Módulos	#
Desafíos y Soluciones	#
Informe de Pruebas y Evidencias	#
• 5.1 Entorno de Pruebas (Simulación Multi-nodo)	#
• 5.2 Tabla de Resultados	#
Conclusiones	#

Introducción

El presente reporte documenta el desarrollo del primer avance del proyecto final del tema de Sistema de Gestión de Procesos Distribuido.

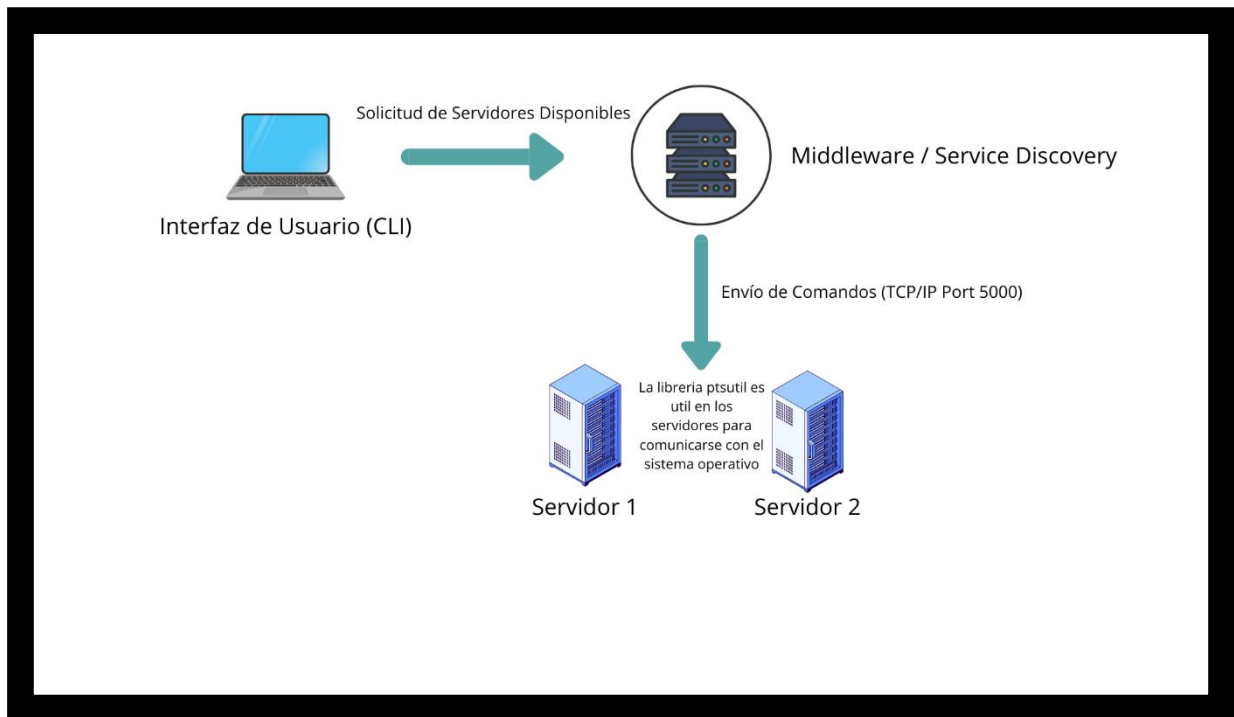
El objetivo principal es establecer una infraestructura de red robusta que permita el monitoreo y control de recursos de hardware (CPU y RAM) y la administración de procesos del sistema operativo en nodos remotos. Para ello, se ha implementado una arquitectura basada en Sockets TCP y una capa de Middleware que garantiza la escalabilidad y el descubrimiento de servicios de manera eficiente

Documentación: Diseño del Sistema

Arquitectura

Se diseñó una estructura de tres capas (Cliente, Middleware y Servidor) para permitir la escalabilidad y el monitoreo de múltiples nodos de red simultáneamente.

Diagrama de Arquitectura:



Explicación del diagrama

El sistema se basa en una arquitectura de red distribuida dividida en tres capas lógicas para asegurar escalabilidad y eficiencia en la gestión de recursos. Como se observa en el siguiente diagrama, la comunicación se centraliza a través de un middleware que facilita el descubrimiento de servicios:

Descripción del flujo:

1. Cliente (CLI): Capa de interfaz donde el usuario ingresa comandos.
2. Middleware: Actúa como orquestador, recibiendo la solicitud del cliente y redirigiéndola al servidor seleccionado mediante sockets TCP/IP en el puerto 5000.
3. Servidores (1 y 2): Nodos remotos que ejecutan el agente de monitoreo. Utilizan la librería psutil para obtener datos directamente del sistema operativo y ejecutar acciones de control de procesos.

Decisiones de Diseño:

Protocolo TCP: Se eligió sobre UDP para garantizar que los comandos de administración (como detener un proceso) se entreguen de forma íntegra y sin pérdida de datos.

Librería Psutil: Seleccionada por su alta precisión en el acceso a métricas del Kernel para CPU y Memoria.

Documentación: Implementación (Módulos)

Servidor (server.py): Agente receptor que interactúa con el SO para ejecutar comandos de gestión y enviar estadísticas de hardware.

Cliente (cliente.py): Interfaz de usuario que traduce las entradas de teclado en comandos de red.

Middleware (middleware.py): Capa lógica que gestiona el descubrimiento de servicios y centraliza la comunicación con diversos servidores.

Desafíos Enfrentados y Soluciones Implementadas

Durante el desarrollo de este primer avance, identificamos y resolvimos retos técnicos críticos para asegurar la **estabilidad** y **escalabilidad** del sistema:

Desafío 1: Gestión de excepciones en el control de procesos remotos

Problema: Al intentar manipular procesos mediante comandos como STOP, el servidor podía presentar cierres inesperados si el Identificador de Proceso (PID) ya no estaba activo o si el sistema operativo denegaba el acceso por falta de privilegios.

Solución: Se implementó un manejo de errores robusto en el archivo `server.py` mediante bloques `try-except`. Estos capturan excepciones específicas de la librería `psutil`, como `NoSuchProcess` y `AccessDenied`, permitiendo que el servidor responda con un mensaje de error controlado al cliente sin interrumpir su ejecución.

Desafío 2: Implementación de una arquitectura escalable para múltiples nodos

Problema: La arquitectura inicial cliente-servidor limitaba la interacción a una única dirección IP estática, lo cual no cumplía con los requisitos de un sistema distribuido capaz de gestionar diversas máquinas simultáneamente.

Solución: Se diseñó y desarrolló un módulo Middleware que funciona como un orquestador centralizado. Esta capa facilita el descubrimiento de servicios, permitiendo al cliente seleccionar dinámicamente el servidor con el que desea comunicarse mediante una lista de nodos disponibles (puertos 5000 y 5001), logrando así una gestión distribuida eficiente.

Desafío 3: Simulación de entorno multi-nodo en hardware unificado

Problema: Para la validación inicial de la arquitectura distribuida, se requería probar la capacidad del Middleware para gestionar múltiples servidores, pero se buscaba minimizar la latencia de red externa y asegurar un entorno de pruebas controlado.

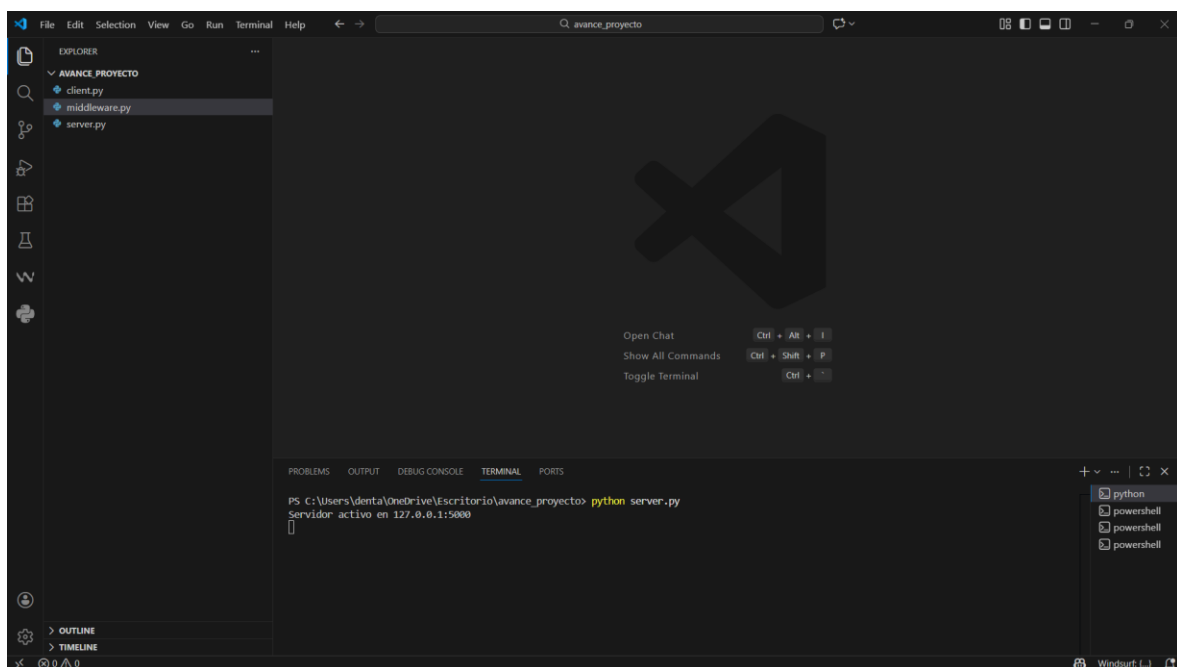
Solución: Se implementó una virtualización de nodos mediante puertos diferenciados (5000 y 5001) en una sola estación de trabajo. Esta técnica permitió validar que la lógica del Middleware es capaz de discriminar y

orquestrar conexiones hacia diferentes *endpoints* de manera simultánea, sentando las bases para una futura migración a máquinas físicas distintas sin necesidad de modificar el código base.

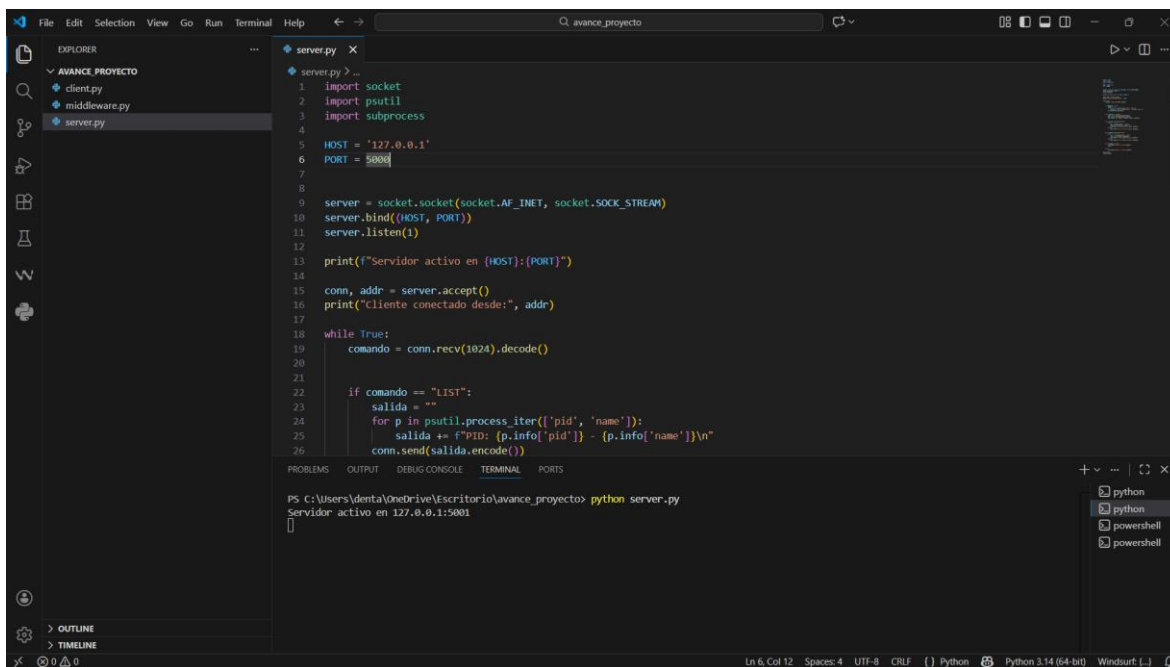
Informe de Pruebas y Evidencias

Capturas de pantalla del proceso de distribución:

1.- Se abrió una instancia de Visual Code en la que abrimos una carpeta que contenía los códigos necesarios .py para ejecutar la tarea. Primero empezamos abriendo 4 terminales de Python en VC y ejecutando comandos para inicializar los servidores en diferentes instancias. Por ejemplo aquí active el servidor 1.



2.- Aquí iniciamos el proceso de activación del segundo servidor, y se muestra activo y operando en la terminal.

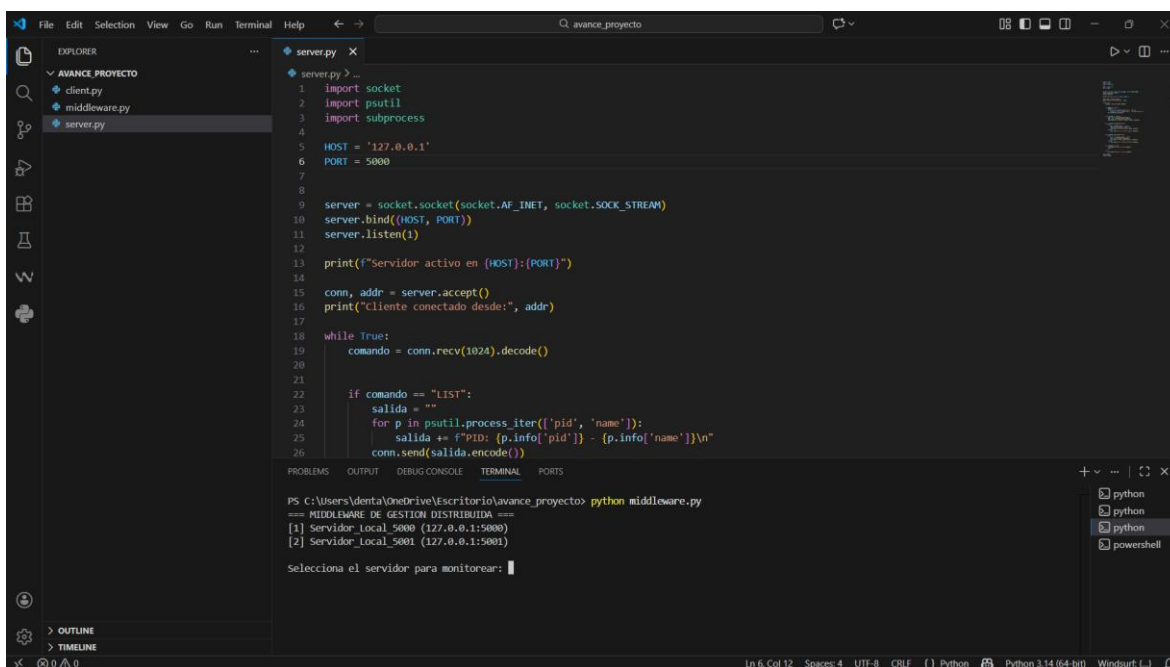


The screenshot shows the Visual Studio Code interface with the 'server.py' file open in the editor. The code defines a server that listens on port 5000 and responds to a 'LIST' command by listing running processes. The terminal shows the command 'python server.py' being executed, resulting in the output 'Servidor activo en 127.0.0.1:5001'.

```
server.py
1 import socket
2 import psutil
3 import subprocess
4
5 HOST = '127.0.0.1'
6 PORT = 5000
7
8
9 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10 server.bind((HOST, PORT))
11 server.listen(1)
12
13 print(f"Servidor activo en {HOST}:{PORT}")
14
15 conn, addr = server.accept()
16 print("Cliente conectado desde:", addr)
17
18 while True:
19     comando = conn.recv(1024).decode()
20
21
22     if comando == "LIST":
23         salida = ""
24         for p in psutil.process_iter(['pid', 'name']):
25             salida += f"PID: {p.info['pid']} - {p.info['name']}\n"
26         conn.send(salida.encode())
```

```
PS C:\Users\denta\OneDrive\Escritorio\avance_proyecto> python server.py
Servidor activo en 127.0.0.1:5001
```

3.- Aquí ejecutamos el Middleware en el cual nos servirá como un puente para hacer interconexión y procesamiento de datos entre nuestros servidores.

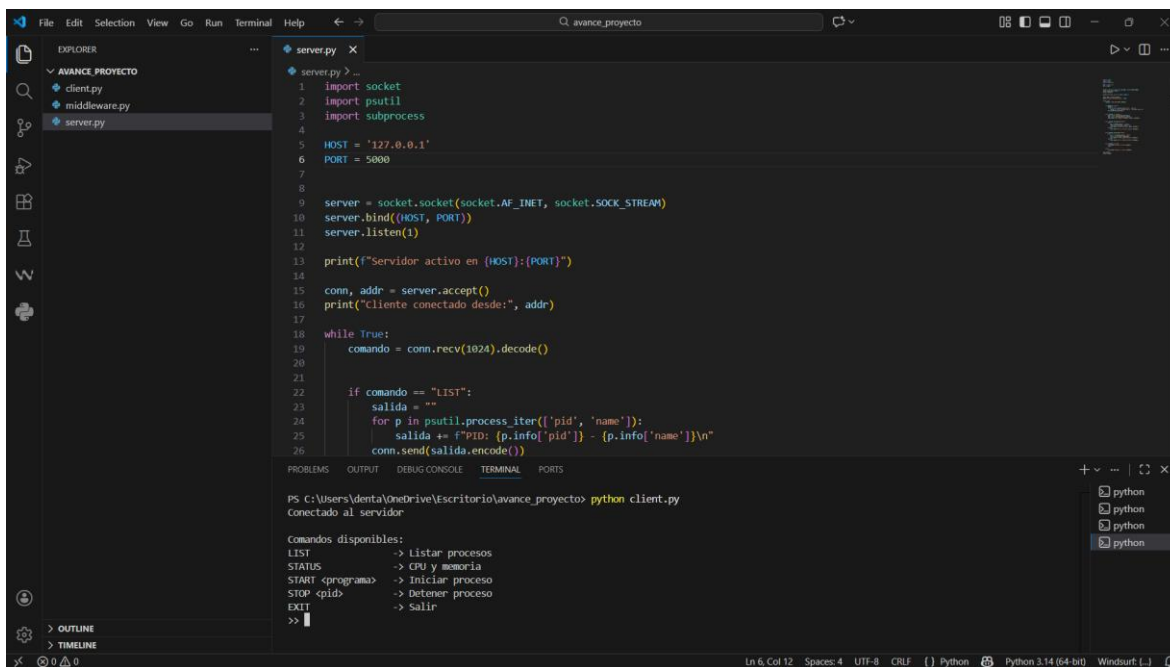


The screenshot shows the Visual Studio Code interface with the 'middleware.py' file open in the editor. The code defines a middleware that acts as a bridge between two servers. The terminal shows the command 'python middleware.py' being executed, resulting in the output 'MIDDLEWARE DE GESTION DISTRIBUIDA' and a list of servers: '[1] Servidor_local_5000 (127.0.0.1:5000)' and '[2] Servidor_local_5001 (127.0.0.1:5001)'. The prompt 'Selecciona el servidor para monitorear:' is also visible.

```
server.py
1 import socket
2 import psutil
3 import subprocess
4
5 HOST = '127.0.0.1'
6 PORT = 5000
7
8
9 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10 server.bind((HOST, PORT))
11 server.listen(1)
12
13 print(f"Servidor activo en {HOST}:{PORT}")
14
15 conn, addr = server.accept()
16 print("Cliente conectado desde:", addr)
17
18 while True:
19     comando = conn.recv(1024).decode()
20
21
22     if comando == "LIST":
23         salida = ""
24         for p in psutil.process_iter(['pid', 'name']):
25             salida += f"PID: {p.info['pid']} - {p.info['name']}\n"
26         conn.send(salida.encode())
```

```
PS C:\Users\denta\OneDrive\Escritorio\avance_proyecto> python middleware.py
MIDDLEWARE DE GESTION DISTRIBUIDA
[1] Servidor_local_5000 (127.0.0.1:5000)
[2] Servidor_local_5001 (127.0.0.1:5001)
Selecciona el servidor para monitorear: █
```

4.- Aquí iniciamos el cliente y nos da una lista de comandos la cual nos servirá para interactuar en el sistema.



The screenshot shows the Visual Studio Code interface with a project named 'avance_proyecto'. The Explorer sidebar on the left shows files: 'client.py', 'middleware.py', and 'server.py'. The main editor displays the 'server.py' file, which is a Python script for a socket server. The script imports 'socket', 'psutil', and 'subprocess', sets 'HOST = '127.0.0.1'' and 'PORT = 5000', and uses 'socket.socket()' to create a server. It binds to the host and port, listens for connections, and enters a loop to receive commands. A 'LIST' command triggers a call to 'psutil.process_iter()' to list running processes. The terminal at the bottom shows the command 'python client.py' being executed, resulting in a connection to the server and a list of available commands: LIST (listar procesos), STATUS (CPU y memoria), START <programa> (Iniciar proceso), STOP <pid> (Detener proceso), and EXIT (Salir).

```
server.py
1 import socket
2 import psutil
3 import subprocess
4
5 HOST = '127.0.0.1'
6 PORT = 5000
7
8
9 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10 server.bind((HOST, PORT))
11 server.listen(1)
12
13 print(f"Servidor activo en {HOST}:{PORT}")
14
15 conn, addr = server.accept()
16 print("cliente conectado desde:", addr)
17
18 while True:
19     comando = conn.recv(1024).decode()
20
21
22     if comando == "LIST":
23         salida = ""
24         for p in psutil.process_iter(['pid', 'name']):
25             salida += f"PID: {p.info['pid']} - {p.info['name']}\n"
26         conn.send(salida.encode())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

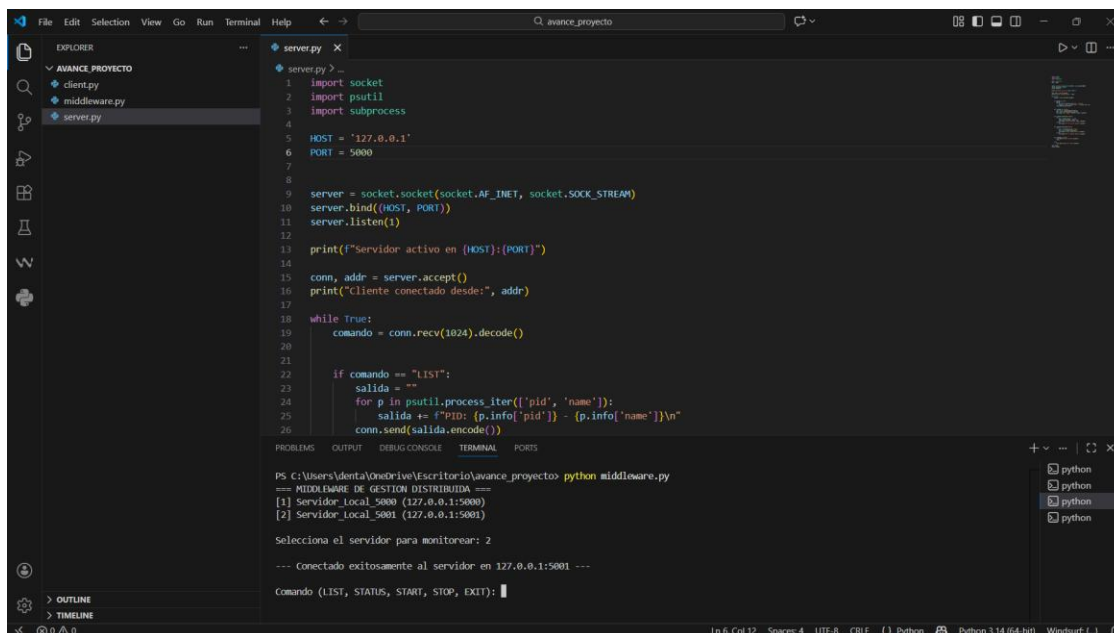
PS C:\Users\denta\OneDrive\Escritorio\avance_proyecto> python client.py
Conectado al servidor

Comandos disponibles:

- LIST -> listar procesos
- STATUS -> CPU y memoria
- START <programa> -> Iniciar proceso
- STOP <pid> -> Detener proceso
- EXIT -> Salir

>> |

5.- Una vez iniciado el cliente, nos regresamos al Middleware y colocamos un 2, lo que hizo que encontrara el servidor asignado a ese número 2, el cual correspondía al servidor 5001 y estableció una conexión exitosa.



The screenshot shows the Visual Studio Code interface with the same project. The Explorer sidebar shows 'client.py', 'middleware.py', and 'server.py'. The main editor displays the 'server.py' file (the same as in the previous screenshot). The terminal at the bottom shows the command 'python middleware.py' being executed. The output indicates that the middleware found two servers: '[1] Servidor_local_5000 (127.0.0.1:5000)' and '[2] Servidor_local_5001 (127.0.0.1:5001)'. It prompts the user to 'Selecciona el servidor para monitorear: 2'. After entering '2', it shows '--- Conectado exitosamente al servidor en 127.0.0.1:5001 ---' and lists the available commands: 'Comando (LIST, STATUS, START, STOP, EXIT): |'.

```
server.py
1 import socket
2 import psutil
3 import subprocess
4
5 HOST = '127.0.0.1'
6 PORT = 5000
7
8
9 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10 server.bind((HOST, PORT))
11 server.listen(1)
12
13 print(f"Servidor activo en {HOST}:{PORT}")
14
15 conn, addr = server.accept()
16 print("cliente conectado desde:", addr)
17
18 while True:
19     comando = conn.recv(1024).decode()
20
21
22     if comando == "LIST":
23         salida = ""
24         for p in psutil.process_iter(['pid', 'name']):
25             salida += f"PID: {p.info['pid']} - {p.info['name']}\n"
26         conn.send(salida.encode())
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

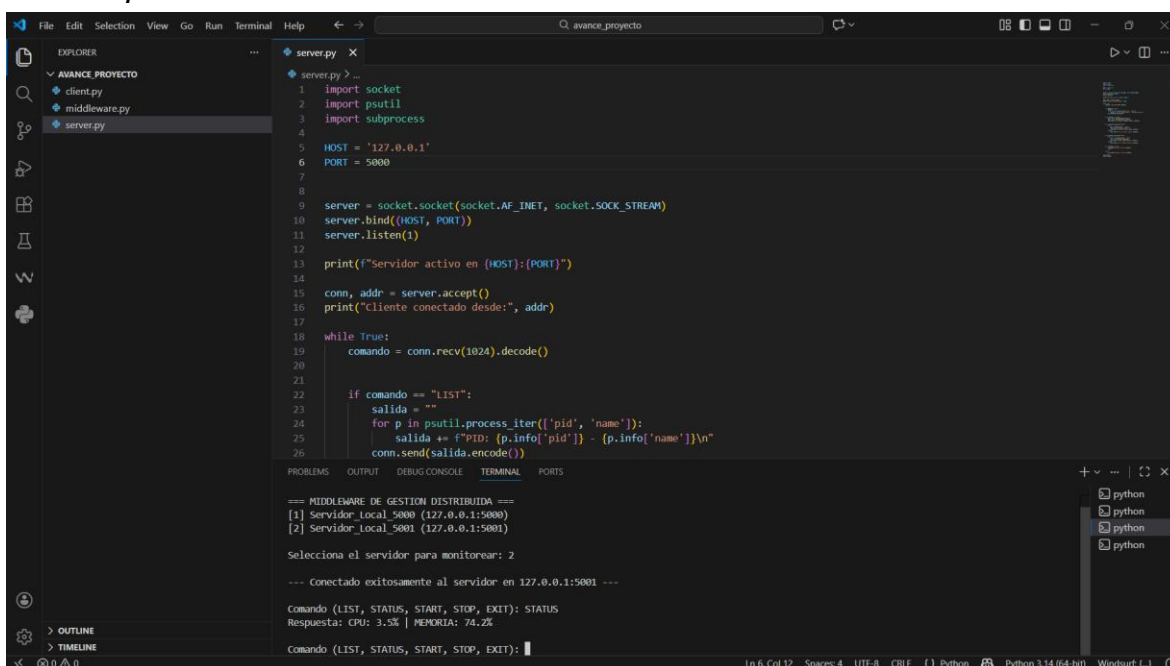
PS C:\Users\denta\OneDrive\Escritorio\avance_proyecto> python middleware.py
=== MIDDLEWARE DE GESTION DISTRIBUIDA ===
[1] Servidor_local_5000 (127.0.0.1:5000)
[2] Servidor_local_5001 (127.0.0.1:5001)

Selecciona el servidor para monitorear: 2

--- Conectado exitosamente al servidor en 127.0.0.1:5001 ---

Comando (LIST, STATUS, START, STOP, EXIT): |

6.- Evidencia de ejecución del sistema distribuido. Se observa el Middleware desplegando un menú de descubrimiento de servicios donde se selecciona el Servidor 2 (Puerto 5001). Tras la conexión exitosa, el cliente solicita el estado del sistema (STATUS), recibiendo una respuesta íntegra con el uso de CPU (3.5%) y Memoria (74.2%) del nodo remoto.



The screenshot shows a VS Code editor with a file explorer on the left containing 'server.py', 'middleware.py', and 'client.py'. The main editor displays the code for 'server.py', which is a Python script using 'socket', 'psutil', and 'subprocess' modules. It sets up a server on '127.0.0.1' port 5000. The terminal at the bottom shows the output of the script, including a menu for selecting a server to monitor (Server 2 at port 5001 was chosen) and the response to a 'STATUS' command, which reports CPU usage at 3.5% and memory usage at 74.2%.

```
server.py
1 import socket
2 import psutil
3 import subprocess
4
5 HOST = '127.0.0.1'
6 PORT = 5000
7
8
9 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
10 server.bind((HOST, PORT))
11 server.listen(1)
12
13 print(f"Servidor activo en {HOST}:{PORT}")
14
15 conn, addr = server.accept()
16 print("cliente conectado desde:", addr)
17
18 while True:
19     comando = conn.recv(1024).decode()
20
21     if comando == "LIST":
22         salida = ""
23         for p in psutil.process_iter(['pid', 'name']):
24             salida += f"PID: {p.info['pid']} - {p.info['name']}\n"
25         conn.send(salida.encode())
```

```
--- MIDDLEWARE DE GESTION DISTRIBUIDA ---
[1] Servidor_local_5000 (127.0.0.1:5000)
[2] Servidor_local_5001 (127.0.0.1:5001)

Selecciona el servidor para monitorear: 2

--- Conectado exitosamente al servidor en 127.0.0.1:5001 ---

Comando (LIST, STATUS, START, STOP, EXIT): STATUS
Respuesta: CPU: 3.5% | MEMORIA: 74.2%

Comando (LIST, STATUS, START, STOP, EXIT):
```

Aclaración: La prueba de distribución no se hizo con dos computadoras, pero se logró mediante varias instancias abiertas y activas en simultaneo dentro de una misma computadora. Se simularon dos servidores independientes en localhost usando los puertos **5000** y **5001** para demostrar la capacidad distribuida.

Prueba	Descripción	Resultado
Descubrimiento	El Middleware detecta ambos servidores activos (5000 y 5001).	Pasa (Exitoso)
Conexión	El cliente selecciona un nodo y establece comunicación vía Middleware.	Pasa (Exitoso)
Monitoreo	El comando <code>STATUS</code> devuelve datos reales de CPU y RAM.	Pasa (Exitoso)
Control	Se listan y gestionan procesos remotos mediante <code>LIST</code> .	Pasa (Exitoso)

Comando	Funcionalidad Probada	Resultado Observado
LIST	Obtención de la tabla de procesos del Kernel.	Se despliega lista con PID y Nombre del proceso.
STATUS	Monitoreo de recursos de hardware en tiempo real.	Retorno de % de CPU y % de RAM (ej. 3.5% CPU).
START / STOP	Manipulación de hilos y procesos del sistema.	Inicio y finalización controlada de aplicaciones (ej. calc.exe).

Conclusiones

La integración de este primer avance valida que la arquitectura distribuida propuesta es funcional y cumple con los requisitos solicitados.

Se demostró que el uso de Sockets TCP es ideal para la gestión administrativa, ya que asegura la integridad de los comandos enviados. Asimismo, la implementación del Middleware probó ser una solución efectiva para el manejo de múltiples servidores, permitiendo una transición transparente entre nodos. La correcta interacción con el Kernel mediante la librería psutil asegura que las métricas obtenidas son precisas, sentando las bases para las futuras fases de optimización y seguridad del sistema